

Vector Space

THE PROBLEM

In the last lesson, you watched a token travel through the layers, its numbers changing at every step. By the end it has a brand-new set of numbers, one of a kind, that doesn't match the vector of any other token.

So how does the model know what this token means, when its numbers are unique and match no token's embedding?

To see the answer, go back to the taste test from Embeddings, where Pepsi looked almost exactly like Coke until a Citrus dimension finally told them apart.

Taste Profile - Coke vs. Pepsi vs. Coffee								
TOKEN	TOKEN ID	DIMENSIONS						
		SWEET	BITTER	FIZZ	HEAT	CAFFEINE	DARK	CITRUS
 Coke	24317	9	1	10	2	3	8	1
 Pepsi	38106	9	1	10	2	3	8	10
 Coffee	51820	1	9	0	9	8	10	0

Notice how Coke and Pepsi's vectors (their rows of numbers) sit much closer to each other than either does to Coffee.

This matters for AI. The model measures closeness with **distance**: how far apart two vectors are when you add up the gap in every dimension, not just a count of which slots differ. The model has established that Coke is more like Pepsi than like Coffee.



MEANING IS A POSITION

Now back to that one-of-a-kind token. The model can't look its numbers up, so it does what we just did with the drinks: it reads the token's **position**. This is the idea of **vector space**: a map where every vector sits somewhere, and similar words share a neighborhood.

Let's demonstrate this by using the example in the Transformer lesson.

The

cat

sat

on

the

mat

during

the

May

rainstorm

because

it

was

tired

IT is a pronoun, so on its own its vector is ambiguous and certainly doesn't mean **CAT**. But as the token runs through the layers, the model uses the surrounding words to work out what **IT** refers to. By the end, its numbers land closer to **CAT** than to any other word in the sentence — the model has figured out that **IT** means the cat.

